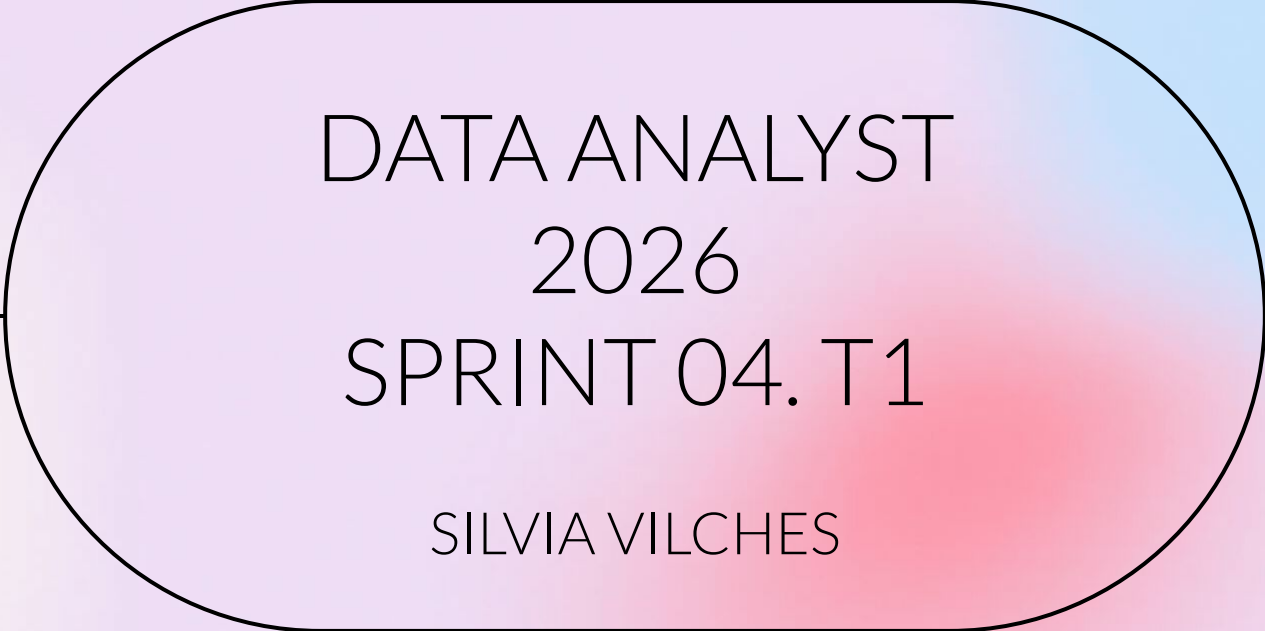




DATA ANALYST

2026

SPRINT 04. T1

SILVIA VILCHES

ÍNDICE



Pág. 04 - 09



Pág. 10 - 14



Pág. 15- 21



1

Ejercicio 1

- » 1. Escribe la consulta que une (**JOIN**) *transactions* y *companies*.
- » 2. Filtra los resultados por la fecha indicada y por país "Alemania". * Fecha: 12 de marzo de 2022
- » 3. Sin ejecutar la consulta, realiza un "**Dry run**" (auditoría de costes)

Si comparamos con consultas y ejercicios anteriores, debido a los MB procesados, esta consulta leerá las tablas completas.

```
25 SELECT *
26 FROM `sprint3_silver.transactions_clean` AS transactions
27 JOIN `sprint3_silver.companies_clean` AS companies
28 USING(business_id)
29 WHERE companies.country = "Germany" AND DATE(timestamp) = "2022-03-12";
```

✓ Esta consulta procesará 10.88 MB cuando se ejecute.

Aunque el coste puede no parecer muy alto en una sola consulta, si ésta deciden hacerla cada día varias personas del mismo o diferentes departamentos, a largo plazo puede suponer un coste inaceptable para la compañía.

1

Ejercicio 2

Paso 1: Generación de datos recientes (datos simulados): Crea una tabla intermedia llamada *sprint3silver.transactions_recent* a partir de la tabla *sprint3_silver.transactions_clean*.

Tu objetivo es conservar todas las columnas, pero sustituir la marca de tiempo original por una nueva generada aleatoriamente, de modo que se sitúe dentro de los últimos 50 días.

```

4 CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
5 SELECT *
6 EXCEPT(timestamp),
7 TIMESTAMP_SUB(
8   CURRENT_TIMESTAMP(),
9   INTERVAL CAST(RAND() * 50 AS INT64) DAY
10 ) AS timestamp
11 FROM `sprint3_silver.transactions_clean`;

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

❗ Esta declaración creó una nueva tabla con el nombre transactions_recent.

La nueva tabla ha sido creada.

▼	📊 sprint3_silver	☆	⋮
	📊 companies_clean	☆	⋮
	📊 credits_cards_clean	☆	⋮
	📊 products_clean	☆	⋮
	📊 transactions_clean	☆	⋮
	📊 transactions_recent	☆	⋮
	📊 users_combined	☆	⋮

• **CREATE OR REPLACE TABLE** – definimos crear la tabla

• **SELECT *** – Seleccionamos todos los campos de la tabla

• **EXCEPT(timestamp)** – excepto el campo “timestamp”

• **TIMESTAMP_SUB** – extrae unidades de tiempo que después definiremos como DAY (por día)

• **CURRENT_TIMESTAMP** – nos devuelve las fechas actuales, pero definiremos cómo queremos que nos las devuelva

• **INTERVAL CAST(RAND() * 50 AS INT64) DAY** – genera valores aleatorios de entre 0 y 50 que en este caso hemos forzado con CAST para que sean valores de tipo numérico.

• **AS timestamp** – le damos nombre al campo nuevo.

Comprobamos las fechas.

✓	SELECT timestamp
11	FROM `sprint3analyticssilver.sprint3_silver.transactions_recent`
12	LIMIT 10;

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON
Fila	timestamp		
1	2026-04-12 09:45:32.199281 UTC		
2	2026-04-12 09:45:32.199281 UTC		
3	2026-04-12 09:45:32.199281 UTC		
4	2026-04-02 09:45:32.199281 UTC		

1

Ejercicio 2

Paso 2: Creación de la tabla optimizada: Crea la tabla final `sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`. Debes elaborar la instrucción **DDL** para configurar la tabla con las siguientes estrategias de almacenamiento físico:

- » **Partición:** Divide la tabla según la fecha del campo `DATE(timestamp)`.
- » **Agrupación:** Ordena los datos dentro de cada partición por "`business_id`".

```

1 CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
2 PARTITION BY DATE(timestamp)
3 CLUSTER BY business_id
4 OPTIONS (description = "a table clustered by business_id & partitioned by date")
5 AS
6 SELECT *
7 FROM `sprint3_silver.transactions_recent`;

```

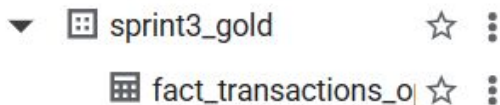
✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

❗ Esta declaración creó una nueva tabla con el nombre `fact_transactions_optimized`.



- **CREATE OR REPLACE TABLE** ``sprint3_gold.fact_transactions_optimized`` – Con un comando DDL (Lenguaje de Definición de Datos) `CREATE`, ordenamos crear la nueva tabla
- **PARTITION BY DATE(timestamp)** – especificamos la columna de la tabla por la cual queremos que haga la partición.
- **CLUSTER BY business_id** – especificamos el campo por el cual vamos a agrupar esta tabla. Los datos de la tabla se organizan según el `business_id`.
- **OPTIONS** (`description = "a table clustered by business_id & partitioned by date"`) – añadimos una descripción de la tabla para en un futuro saber qué o por qué se ha hecho.
- **AS SELECT * FROM** ``sprint3_silver.transactions_recent``; – desde la tabla que ya teníamos creada en el ejercicio anterior.

Necesitamos que nuestras consultas filtren por un periodo de tiempo, por lo tanto vamos a particionar la tabla según la fecha. Con esto, haremos que los costos de nuestra empresa bajen considerablemente cuando en CM de Alemania filtre por día las transacciones.

Para reducir la cantidad de datos que procesará cada consulta organizaremos los valores de la tabla con id de la compañía.

1

Ejercicio 2

Paso 2: Creación de la tabla optimizada: Crea la tabla final *sprint3_gold.fact_transactions_optimized* a partir de los datos recientes que acabas de generar en *transactions_recent*. Debes elaborar la instrucción **DDL** para configurar la tabla con las siguientes estrategias de almacenamiento físico:

» **Partición:** Divide la tabla según la fecha del campo *DATE(timestamp)*.

» **Agrupación:** Ordena los datos dentro de cada partición por “*business_id*”.

DETALLES

sprint3analyticssilvia / Datasets / sprint3_gold / Tables / fact_transactions_optimized	
☆ fact_transacti...	Consulta Chat Abrir en ▾ Compartir ▾
i Esta es una tabla particionada. Learn more	
Esquema	Detalles Vista previa Explorador de tablas Vista previa Estadística
predeterminado	
No distingue mayúsculas de minúsculas	false
Descripción	a table clustered by business_id & partitioned by date
Etiquetas	
Claves primarias	
Etiquetas	
Tipo de tabla	Particionada
Particionada por	DAY
Particionada en el campo	timestamp
Vencimiento de la partición	60 días
Filtro de partición	No es obligatorio
Agrupada por	business_id

1

Ejercicio 3

- 🧩 Crea una consulta **SQL** que seleccione todas las columnas (**SELECT ***) y filtre los datos de los últimos 30 días.
- » **Paso 1 (tabla no optimizada):** Dirige tu consulta a la tabla *sprint3_silver.transactions_recent*. Observa el validador.
 - » **Paso 2 (tabla optimizada):** Sin cambiar nada más en la lógica de la consulta, cambia únicamente la cláusula **FROM** para que apunte a la tabla particionada *sprint3_gold.fact_transactions_optimized*.

TABLA NO OPTIMIZADA

```
28 SELECT *
29 FROM `sprint3_silver.transactions_recent`
30 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

✓ Esta consulta procesará 10.87 MB cuando se ejecute.

TABLA OPTIMIZADA

```
28 SELECT *
29 FROM `sprint3_gold.fact_transactions_optimized`
30 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

✓ Esta consulta procesará 6.64 MB cuando se ejecute.

38,91 % MENOS DE DATOS PROCESADOS

Al particionar la tabla por *DATE(timestamp)* y agruparla por *business_id*, hemos logrado reducir los bytes procesados de 10,87 MB a 6,64 MB, lo que supone un **ahorro del 38,91 %** en comparación con la consulta en la tabla no optimizada.

En lugar de realizar una lectura completa de toda la tabla, ahora solo lee los últimos 30 días, es decir, ignora el resto de fechas.

En una empresa real, este porcentaje significaría un ahorro significativo a largo plazo.

1

Ejercicio 4



Crea una Vista Materializada llamada *sprint3_gold.mv_daily_sale* que muestre las **Ventas Totales por día**.



Haz una consulta que muestre la vista materializada creada con la consulta ejecutada y los bytes procesados

```

7 CREATE MATERIALIZED VIEW sprint3_gold.mv_daily_sale AS (
8   SELECT DATE(timestamp) AS date,
9   COUNT(transaction_id) AS total_sales
10  FROM `sprint3_gold.fact_transactions_optimized`
11  GROUP BY date
12 );

```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo

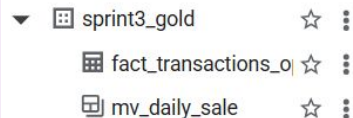
Resultados

Detalles de la ejecución

Gráfico de

❗ Esta declaración creó una nueva vista materializada denominada mv_daily_sale.

La vista ha sido creada correctamente.



```

9
10 SELECT *
11 FROM `sprint3analyticssilvia.sprint3_gold.mv_daily_sale`
12 ORDER BY date DESC;

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

JSON

Fila	date	total_sales
1	2026-05-21	982
2	2026-05-20	1976
3	2026-05-19	2024
4	2026-05-18	2070
5	2026-05-17	2014
6	2026-05-16	2020

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

Hora de creación	21 may 2026, 1:54:40 p.m. UTC+2
Hora de inicio	21 may 2026, 1:54:40 p.m. UTC+2
Hora de finalización	21 may 2026, 1:54:40 p.m. UTC+2
Duración	0 s
Bytes procesados	816 B
Bytes facturados	10 MB
Milisegundos de ranura	50



2

Ejercicio 1



Crea una CTE llamada **VIP_Stats** que agrupe por usuario y calcule:

Gasto total (**SUM**) // Número de transacciones (**COUNT**) // Importe medio por compra (**AVG**), redondeado a dos decimales // Compra máxima (**MAX**) // Filtro: Conserva solo aquellos en los que el gasto total sea > 500.

» Realiza una unión cruzada de la CTE con **users_combined** para obtener los datos personales.

Columnas: **user_id**, **full_name**, **email**, **num_purchases**, **average_ticket**, **max_purchase**, **total_spent**.

Ordenadas por **total_spent** en orden descendente.

```
12 WITH VIP_Stats AS
13 (
14   SELECT transactions_clean.user_id,
15   ROUND(SUM(transactions_clean.amount), 2) AS total_amount_user,
16   COUNT(transactions_clean.transaction_id) AS total_transactions,
17   ROUND(AVG(transactions_clean.amount), 2) AS avg_ticket,
18   MAX(transactions_clean.amount) AS max_purchase,
19   FROM 'sprint3_silver.transactions_clean' as transactions_clean
20   GROUP BY transactions_clean.user_id
21   HAVING total_amount_user > 500)
22 SELECT users_combined.user_id,
23   CONCAT(users_combined.name, ' ', users_combined.surname) AS full_name,
24   users_combined.email,
25   VIP_Stats.total_transactions, VIP_Stats.avg_ticket, VIP_Stats.max_purchase, VIP_Stats.total_amount_user
26 FROM VIP_Stats
27 INNER JOIN 'sprint3_silver.users_combined' AS users_combined
28 USING(user_id)
29 ORDER BY VIP_Stats.total_amount_user DESC;
30
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar los resultados Abrir en

Información del trabajo		Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
Resultados							
File	user_id	full_name	email	total_transactions	avg_ticket	max_purchase	total_amount_user
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.54
2	318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.5
3	455	Fwll Phylgmhnd	fwll.phylgmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatl Rdwejcvoa	fqatl.rdwejcvoa@example.com	78	410.46	675.78	32016.11
5	185	Molly Gilliam	donec@outlook.co.uk	110	275.75	659.31	30333.03
6	285	Brcxpmj Lnyna	brcxpmj.lnyna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgyfrids	sfzzoh.xgyfrids@example.com	81	300.67	604.9	24353.91
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfcldgh	qikcvc.nvfcldgh@example.com	54	351.69	600.77	18991.45
11	388	Rbiuonac Cfvkdx	rbiuonac.cfvkdx@example.com	64	295.67	479.03	18973.1

Definimos la CTE y le damos un nombre con la cláusula **WITH VIP_Stats AS ()** y hacemos los cálculos que nos ha pedido el equipo de marketing con **SUM**, **AVG**, **MAX** y **COUNT** que luego más tarde utilizaremos para combinar con la tabla de usuarios **users_combined**.

Para combinar en una sola columna el nombre y el apellido utilizamos **CONCAT** para juntar dos **STRINGS** en un único valor separado por un espacio " " .

2

Ejercicio 2

Crea una consulta sobre la vista materializada `sprint3_gold.mv_daily_sales` que genere un informe con las siguientes 4 columnas:

- » 1. **Fecha:** La fecha de venta.
- » 2. **Ventas_hoy:** El total vendido ese día.
- » 3. **Ventas_ayer:** El total vendido el día anterior (utilizando funciones de ventana).
- » 4. **Diferencia_porcentual:** El porcentaje de crecimiento o descenso en comparación con el día anterior, redondeado a dos decimales.

```
10 SELECT date,
11    total_sales AS today_sales,
12    LAG(total_sales) OVER (ORDER BY date) AS yesterday_sales,
13    ROUND((total_sales - LAG(total_sales) OVER (ORDER BY date)) / LAG(total_sales) OVER (ORDER BY date) * 100, 2) AS percentage_sale
14 FROM   `sprint3_gold.mv_daily_sale`
15 ORDER BY date DESC;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar los resultados ▾

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	date	today_sales	yesterday_sales	percentage_sale	
1	2026-05-21	982	1976	-50.3	
2	2026-05-20	1976	2024	-2.37	
3	2026-05-19	2024	2070	-2.22	
4	2026-05-18	2070	2014	2.78	
5	2026-05-17	2014	2020	-0.3	
6	2026-05-16	2020	2049	-1.42	

Utilizamos nuestra vista materializada para realizar la consulta como si fuese una tabla cualquiera. `FROM `sprint3_gold.mv_daily_sales``

Con la función de ventana `LAG` tomamos como referencia las ventas total y nos devuelve el valor de la fila anterior, es decir, nos devuelve el total de ventas del día precedente. Pero necesitamos indicarle el orden, gracias a `OVER`, si no, los días estarían desordenados y no podría darnos el resultado del día anterior.

Utilizamos la misma cláusula para la diferencia porcentual, ya que debemos calcular con el día actual de referencia y el anterior.

La fórmula utilizada para calcular esta diferencia que he utilizado es:

$$((\text{ventas del día actual} - \text{ventas del día anterior}) / \text{ventas del día anterior}) * 100$$
 De esta manera, nos puede devolver un valor % en negativo o positivo.

* Podemos ver que en los últimos días ha habido un decrecimiento de ventas constante (excepto el 18/05). Estos valores son bastante estables y no son significativos por lo que pueden significar algo normal según el día de la semana, del mes, etc. Depende del histórico.

Sin embargo, el decrecimiento de -50% del 21/05 sí que resulta preocupante, ya que las ventas se han reducido a la mitad. Se ha de valorar qué ha pasado o qué ha podido influir.

2

Ejercicio 3

✿ Utilizando la vista materializada `mv_daily_sales`, genera un informe con tres columnas:

- » 1. Fecha.
- » 2. Ventas del día: redondeadas a dos decimales.
- » 3. Ventas acumuladas en lo que va de año: un total acumulado de ventas que debe reiniciarse cada 1 de enero. El resultado final debe redondearse a dos decimales.

```

5
6 SELECT date,
7 ROUND(total_sales, 2) AS sales_today,
8 ROUND(SUM(total_sales) OVER (PARTITION BY EXTRACT(YEAR FROM date)
9 ORDER BY date
10 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),
11 2) AS sales_YTD
12 FROM `sprint3_gold.mv_daily_sale`
13 ORDER BY date DESC;
14

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ej
Fila	date	sales_today	sales_YTD	
1	2026-05-21	982.0	100000.0	
2	2026-05-20	1976.0	99018.0	
3	2026-05-19	2024.0	97042.0	
4	2026-05-18	2070.0	95018.0	
5	2026-05-17	2014.0	92948.0	
6	2026-05-16	2020.0	90934.0	

Utilizamos nuestra vista materializada para realizar la consulta como si fuese una tabla cualquiera. `FROM `sprint3_gold.mv_daily_sales`` que ya tiene el cálculo de ventas por día.

Con `SUM` le pedimos que calcule el total de ventas pero necesitamos qué referencia ha de tomar para esta suma, gracias a `OVER`. Aquí vamos a especificarlo.

Primero extraemos el año de cada registro de fecha. `EXTRACT(YEAR FROM date)`. Ordenamos las fechas, si no, no sabría cómo sumar por el año estando desordenados. Con `PARTITION BY` digamos que “agrupa” por año estas fechas.

Al haber “agrupado” por año, con `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` hace el cálculo **ENTRE** el primer registro/día del año (día 1) Y `CURRENT ROW` hasta el día actual (o último día).

*Creo que sin la “pista técnica” hubiese sido muy complicado llegar a esta solución.

2

Ejercicio 4



Genera una lista de los usuarios que hayan realizado (o superado) su tercera compra, indicando:

1. Datos del usuario (user_id, full_name, email).
2. La fecha y el importe exacto de la tercera compra.
3. Media de las tres primeras: el gasto medio de sus transacciones 1, 2 y 3.

```

1 WITH first_transactions AS (
2   SELECT user_id, amount, DATE(timestamp) AS purchase_date,
3   ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS purchase_order
4   FROM 'sprint3_gold.fact_transactions_optimized'
5   QUALIFY purchase_order <= 3),
6
7   third_purchase AS (
8     SELECT user_id, amount, DATE(timestamp) AS purchase_date,
9     ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS purchase_order
10    FROM 'sprint3_gold.fact_transactions_optimized'
11    QUALIFY purchase_order = 3)
12
13 SELECT users_combined.user_id,
14        CONCAT(users_combined.name, ' ', users_combined.surname) AS full_name,
15        users_combined.email,
16        third_purchase.purchase_order, third_purchase.amount,
17        ROUND(AVG(first_transactions.amount), 2) AS avg_first_transactions
18 FROM 'sprint3_silver.users_combined' AS users_combined
19 JOIN first_transactions
20 USING (user_id)
21 JOIN third_purchase
22 USING (user_id)
23 GROUP BY users_combined.user_id, full_name, users_combined.email, third_purchase.purchase_order, third_purchase.amount
24 ORDER BY avg_first_transactions DESC;
25

```

Se completó la consulta

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
Fila	user_id	full_name	email	purchase_order	amount	avg_first_transact...		
1	455	Fwflf Phylgnmhnd	fwflf.phylgnmhnd@example.com	3	678.61	678.61		
2	285	Brcxprnj Lnnpa	brcxprnj.lnnpa@example.com	3	630.52	630.52		
3	2461	Pgmgeu Fbpvblsy	pgmgeu.fbpvblsy@example.com	3	541.56	570.32		
4	86	Quintessa Buckley	justo.sit.amet@hotmail.ca	3	595.53	552.27		
5	382	Dapzqvm Goznmkim	dapzqvm.goznmkim@example.c...	3	568.38	541.2		
6	4895	Lpkmlfp Bbkkekcc	lpkmlfp.bbkkekcc@example.com	3	500.16	536.51		
7	2314	Rutadk Cgujfucf	rutadk.cgujfucf@example.com	3	632.74	534.7		
8	1921	Pnfuow Khufnof	pnfuow.khufnof@example.com	3	562.25	529.75		
9	396	Tymigul Escapyhks	tymigul.escapyhks@example.c...	3	445.45	520.92		
10	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	3	231.04	512.79		
11	259	Slade Downs	nunc@protonmail.net	3	755.0	512.18		
12	4152	Fjiqah Qekpywuj	fjiqah.qekpywuj@example.com	3	520.33	509.58		
13	4704	Cdkmoo Gaqthofe	cdkmoo.gaqthofe@example.com	3	654.58	509.44		
14	362	Rhelz Xwsnyxcd	rhelz.xwsnyxcd@example.com	3	558.08	506.85		

Lo primero que haremos es crear una **CTE** para después realizar la consulta final de una manera más sencilla y menos costosa. Le llamaremos *first_transactions* y crearemos un listado ordenado de las transacciones que agruparemos por id de usuario. Esto lo hacemos con la cláusula **ROW_NUMBER** que ordena de 1 a x. Con **OVER** le indicaremos cómo queremos que haga este listado. Agrupado por "*user_id*" (**PARTITION BY**) y ordenado por fecha (**ORDER BY**). Con **QUALIFY** filtraremos solo las transacciones ordenadas cuyo orden sea igual o menor de 3.

Volvemos a crear otra **CTE** poder filtrar por este *raking* creado. Una vez ordenadas estas transacciones de 1 a 3, necesitamos, tal y como nos pide el ejercicio, solo la tercera: su fecha y su *amount* (**QUALIFY** = 3).

Por último, realizamos la consulta definitiva donde seleccionamos todos los campos que nos piden, los llamamos desde las **CTE**, que combinamos con la tabla *users_combined* donde tenemos los datos de los usuarios. Nos queda calcular la media (**AVG**) de las 3 transacciones que tenemos en la función *first_transactions*. Ordenamos el ticket medio de mayor a menor para que el equipo de marketing sepa cuales con aquellos con mayor calidad y lleven a cabo su campaña..



Ejercicio 1



Crea la tabla «*dim_transactions_flat*» desnormalizando la información. Debes «desglosar» la matriz de productos y cruzar los datos con el catálogo maestro para obtener los nombres y precios individuales.

CÓDIGO DDL

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS (  
2 SELECT transactions.transaction_id,  
3 DATE(transactions.timestamp) AS purchase_date,  
4 transactions.amount,  
5 products.product_id,  
6 products.name AS product_name,  
7 products.price AS product_price  
8 FROM `sprint3analyticssilvia.sprint3_gold.fact_transactions_optimized` AS transactions  
9  
10 CROSS JOIN UNNEST(SPLIT(transactions.product_ids)) AS product_id  
11  
12 INNER JOIN `sprint3analyticssilvia.sprint3_silver.products_clean` AS products  
13 ON products.product_id = CAST(product_id AS INT64)  
14 );
```

✓ Esta consulta procesará 6.1 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

❗ Esta declaración reemplazó la tabla denominada dim_transactions_flat.

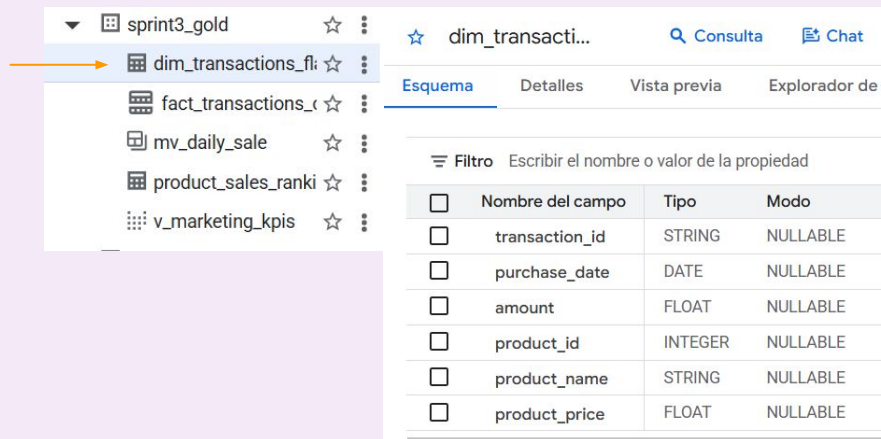
VISTA PREVIA

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje	Perfil de c
Fila	transaction_id	purchase_date	amount	product_id	product_name	product_price	
1	2CAF7179-5E00-4269-9172-AC...	2026-04-01	161.11	1	Direwolf Stannis	161.11	
2	7185E9A9-17CA-4537-86C9-5D...	2026-04-01	388.65	1	Direwolf Stannis	161.11	
3	737F294E-CE12-47E8-9755-A0...	2026-04-01	161.11	1	Direwolf Stannis	161.11	
4	D2C4D6F7-7AA8-42B0-8058-87...	2026-04-01	438.9	1	Direwolf Stannis	161.11	
5	FE32A023-2FAB-4B26-91ED-8C...	2026-04-01	393.24	1	Direwolf Stannis	161.11	
6	68573198-348C-494D-AD9A-CD...	2026-04-01	221.44	1	Direwolf Stannis	161.11	
7	BEF361A5-A7F1-4C28-904C-35...	2026-04-01	294.94	1	Direwolf Stannis	161.11	
8	B5B34DD7-6AF5-4876-8742-6C...	2026-04-01	402.49	1	Direwolf Stannis	161.11	
9	7A9C45B0-0742-4E89-80CE-18...	2026-04-01	502.36	1	Direwolf Stannis	161.11	
10	416FD142-9A1D-434B-B5AD-F8...	2026-04-01	218.36	1	Direwolf Stannis	161.11	
11	384ABEA8-6231-4BC3-A412-CD...	2026-04-01	363.46	1	Direwolf Stannis	161.11	
12	A880B01C-43E5-4F7F-9FDE-25...	2026-04-01	329.6	1	Direwolf Stannis	161.11	
13	1433C1A5-3F14-49DA-85F4-31...	2026-04-01	552.11	1	Direwolf Stannis	161.11	
14	18CDD215-1890-4D48-9BE4-B7...	2026-04-01	391.4	1	Direwolf Stannis	161.11	
15	B2091D34-0697-4426-8E63-A5...	2026-04-01	367.85	1	Direwolf Stannis	161.11	
16	39D8AF98-6F6E-4C11-9774-C9...	2026-04-01	283.84	1	Direwolf Stannis	161.11	

Ejercicio 1

🧩 Crea la tabla «*dim_transactions_flat*» desnormalizando la información. Debes «desglosar» la matriz de productos y cruzar los datos con el catálogo maestro para obtener los nombres y precios individuales.

ESQUEMA DE LA TABLA



The screenshot shows a data catalog interface. On the left, a sidebar displays a tree view of databases and tables. The 'sprint3_gold' database is expanded, showing several tables: 'dim_transactions_flat', 'fact_transactions_c', 'mv_daily_sale', 'product_sales_ranki', and 'v_marketing_kpis'. An orange arrow points to the 'dim_transactions_flat' table. The main panel on the right shows the schema details for 'dim_transactions_flat'. It includes tabs for 'Esquema', 'Detalles', 'Vista previa', and 'Explorador de'. The 'Esquema' tab is active, displaying a table with columns: 'Nombre del campo', 'Tipo', and 'Modo'. The table lists the following fields:

Nombre del campo	Tipo	Modo
transaction_id	STRING	NULLABLE
purchase_date	DATE	NULLABLE
amount	FLOAT	NULLABLE
product_id	INTEGER	NULLABLE
product_name	STRING	NULLABLE
product_price	FLOAT	NULLABLE

```
1 SELECT product_id,product_name,  
2 COUNT(product_id) AS total_purchase  
3 FROM `sprint3_gold.dim_transactions_flat`  
4 GROUP BY product_id, product_name  
5 ORDER BY total_purchase DESC  
6 LIMIT 5;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON
Fila	product_id	product_name	total_purchase	
1	52	riverlands the duel	2654	
2	29	Tully maester Tarly	2635	
3	21	duel Direwolf	2609	
4	16	the duel warden	2608	
5	66	mustafar jinn	2601	

3

Ejercicio 3

- 🧩 Debes actualizar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.
- ⚙️ » 1. Funciones definidas por el usuario (UDF): Crea una función SQL persistente llamada `calculate_tax(amount)` que tome un valor numérico y devuelva el resultado tras aplicar un impuesto del 21 %.

Creemos la función persistente con `CREATE FUNCTION` y le damos el nombre que nos piden. Necesitamos darle un parámetro (que será el precio del producto).

Por último, definimos qué hará esta función necesitamos que nos devuelva (`RETURN`) el valor del parámetro dado + el IVA.

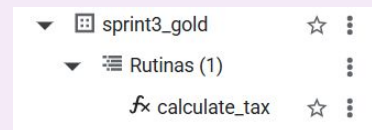
```
1 CREATE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)
2 RETURNS FLOAT64
3 AS (
4   ROUND(amount * 1.21, 2)
5 );
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo **Resultados** Detalles de la ejecución Gráfico de ejecución

❗ La declaración creó una función nueva llamada sprint3analyticssilvia.sprint3_gold.calculate_tax.



Tenemos la **FUNCIÓN** creada.

3

Ejercicio 3

- Debes actualizar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.
2. Modifica el código de creación de la tabla (del ejercicio 1) para utilizar tu nueva función `calculate_tax` y generar una nueva columna: `'product_price_tax_inc'`.
- Configura una consulta programada para que esta instrucción se ejecute automáticamente todos los días a las 07:00 a. m.

ACTUALIZACIÓN DE LA TABLA

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS (
2   SELECT transactions.transaction_id,
3     DATE(transactions.timestamp) AS purchase_date,
4     transactions.amount,
5     products.product_id,
6     products.name AS product_name,
7     products.price AS product_price,
8     `sprint3_gold.calculate_tax`(products.price) AS product_price_tax_inc
9   FROM `sprint3analyticssilvia.sprint3_gold.fact_transactions_optimized` AS transactions
10  CROSS JOIN UNNEST(SPLIT(transactions.product_ids)) AS product_id
11  INNER JOIN `sprint3analyticssilvia.sprint3_silver.products_clean` AS products
12    ON products.product_id = CAST(product_id AS INT64)
13 );
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

Esta declaración reemplazó la tabla denominada `dim_transactions_flat`.

PROGRAMACIÓN DE LA CONSULTA

Exportar como consulta programada

Configuración válida.

Detalles y programación

Nombre de la consulta programada *
TAX update dim_transactions_flat

Opciones de programación

Frecuencia de repetición *
Días

A la(s) *
07:01 UTC

☒ Comenzar ahora ☐ Comenzar a la hora definida

Fecha de inicio y hora de ejecución
26/5/26, 1:45 p.m.

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

Fecha de finalización

Programado para ejecutarse todos los días a las 07:01 UTC a partir de ahora

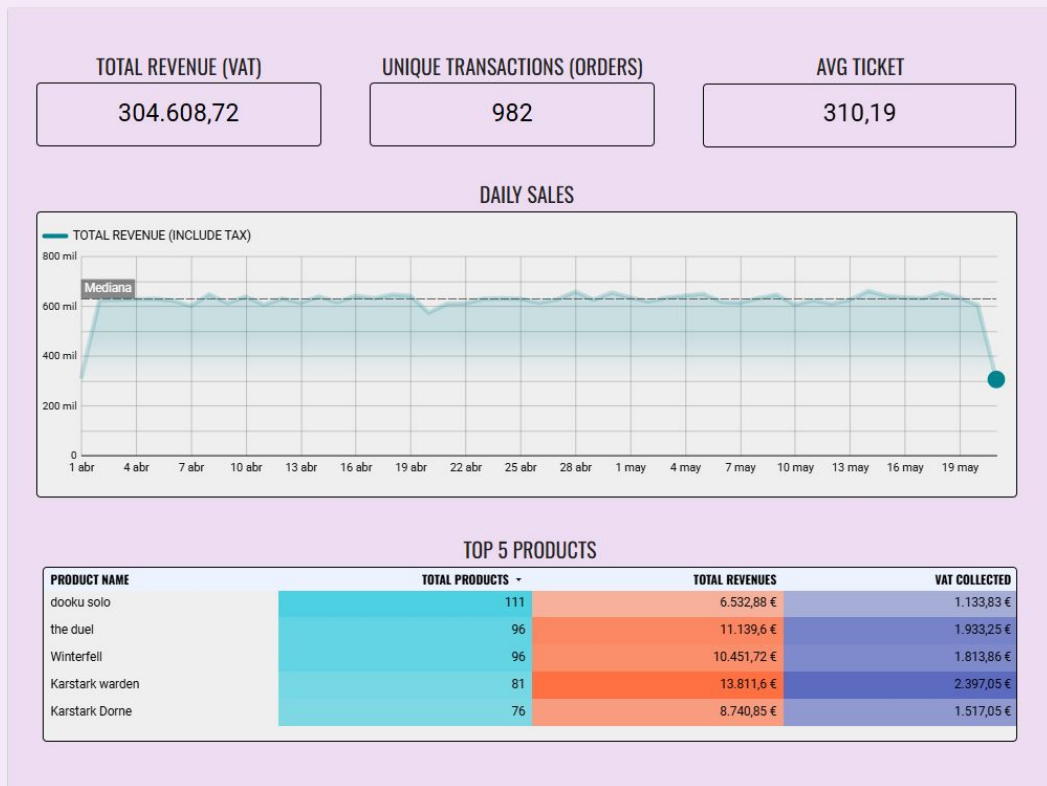
Guardar Cancelar

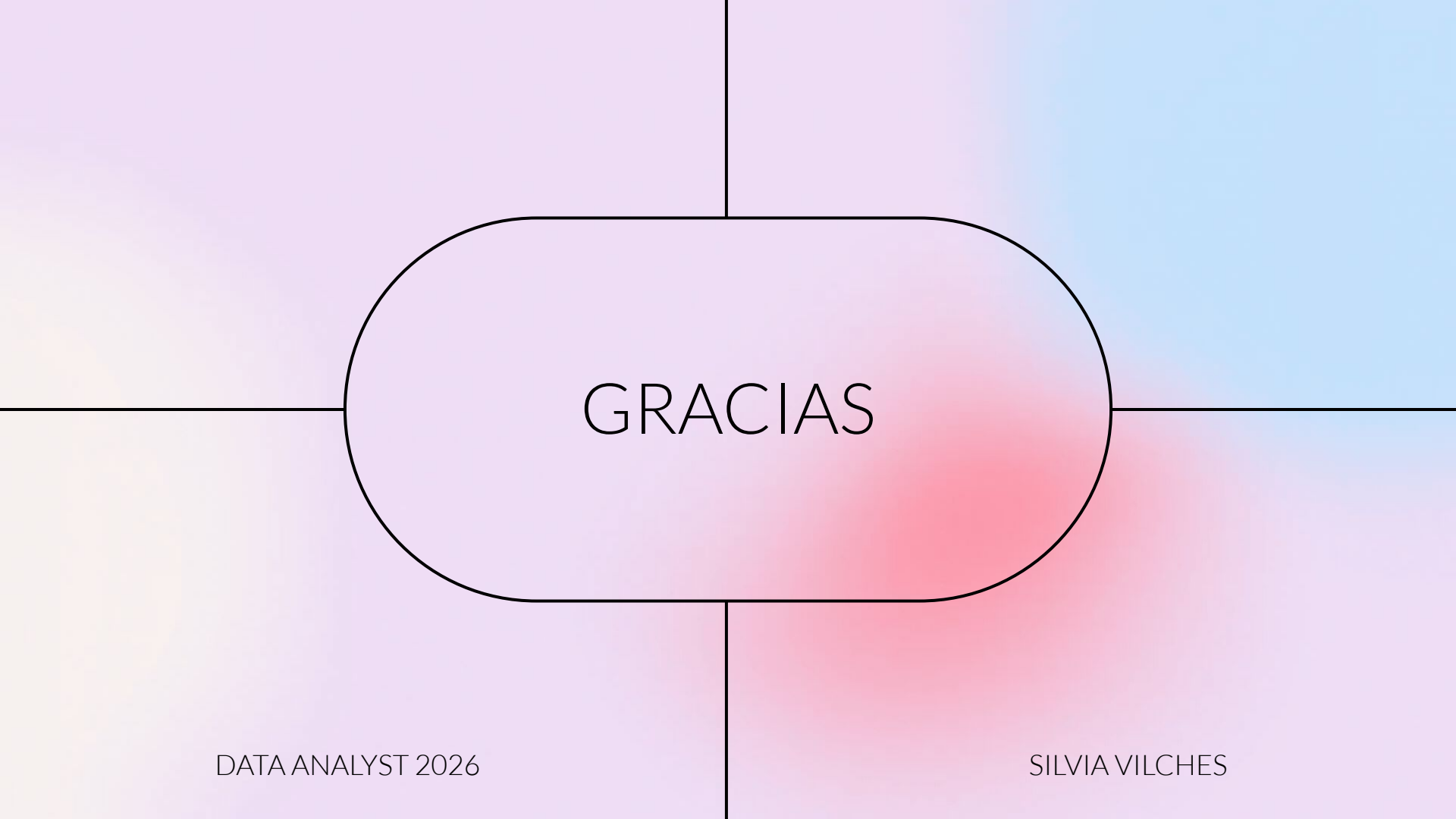
Se recomienda no programar las consultas exactamente a en punto (por ejemplo, a las 07:00 a.m.), ya que en ocasiones esto puede provocar activaciones múltiples y duplicar datos. Es recomendable programarlas unos minutos antes o después, por esta razón he marcado las 07:01 a.m.

Ejercicio 3

- Debes actualizar tu tabla `dim\_transactions\_flat` para incluir el cálculo de impuestos y automatizar su regeneración.
3. Conecte **Looker Studio** a su tabla `dim\_transactions\_flat` y cree un panel: «**Monitor de rendimiento de ventas**»

<https://datastudio.google.com/reporting/18fc9cf4-c15d-4746-b916-53376781c6e5/page/EJQzF/edit>





GRACIAS

DATA ANALYST 2026

SILVIA VILCHES